

Integrity-Aware Autonomous Navigation for UAVs

Stage 1 Report: Architecture, Modules, and Contributions

Integrity-Aware Autonomous Navigation with PL Field Predictor

[Your Name]

Supervisor: [Advisor Name]

May 8, 2026

[Laboratory / Department]

Outline

Ch1 Background and Core Problem

Ch2 Overall Architecture and Design Principles

Ch3 Localization Module: GNSS+LiDAR+IMU Tightly Coupled FGO

Ch4 Dual-Source ARAIM

(Core Contribution 1)

Ch5 PL Field Predictor

(Core Contribution 2)

Ch6 PI Function and Two-Stage Cost Architecture (Core Contribution 3)

Ch7 Planning Module: EGO-Planner Extension

Ch8 Experimental Validation

Ch9 Summary and Next Steps

Ch1 Background and Core Problem

1.1 Background: Integrity Requirements for Safety-Critical Autonomy

Status quo of the three-layer autonomy stack

- **Estimation:** VIO / LIO / GNSS-INS fusion can already provide $\hat{x}_k$ and covariance Σ_k
- **Planning:** EGO-Planner, Fast-Planner consume **mean estimates** $\hat{x}_k$ and are largely unaware of covariance and faults
- **Control:** Geometric control / MPC are mature

New requirements in safety-critical scenarios

Urban low-altitude logistics, inspection, and emergency rescue $\Rightarrow$ require not only **high accuracy**, but also a **known error bound** (bounded error guarantee)

Aviation lesson: ARAIM

GNSS already has a mature **Advanced RAIM** framework:

$$P(|\epsilon| > \text{PL}) \leq \text{IR}_{\text{req}} \approx 10^{-7}/h$$

but it is not yet standardized for UAV multi-sensor fusion.

Position of this work

Upgrade ARAIM *protection level (PL)* from a **monitoring tool** to a **planning metric**.

1.2 Three Key Research Gaps

Gap	Current status	Strategy in this work
Information flow	SLAM covariance Σ_k is discarded by the planner; the planner uses only the mean state to construct ESDF collision costs	Introduce PL as a first-class SLAM-to-planner metric
Methodology	ARAIM is mature for GNSS-only systems; GNSS+LiDAR+IMU has no agreed fault tree	Propose dual-source ARAIM : GNSS fault atom + LiDAR factor-block fault atom, with shared statistical semantics
Spatiotemporal dimension	ARAIM gives pointwise PL at the current time and position ; the planner needs a PL field at future times and candidate positions	Design PL Field Predictor : a local 3D scalar field $\mathbf{PL}(p)$, queried by A* and B-spline optimization

1.3 Core Research Question and Scope of This Report

Core question (one sentence)

How can the protection level produced by the localization module become a first-class planning signal, so that a UAV actively avoids low-integrity regions while preserving real-time performance and certifiability?

Stage 1 scope (this report)

- ✓ Discrete-time FGO + decoupled planning architecture
- ✓ Dual-source ARAIM (GNSS + LiDAR VGICP factor block)
- ✓ PL Field Predictor + PI cost injection into EGO-Planner
- ✓ **ROS2-based SO3 UAV simulation** with EGO-Planner, SO3 controller, GNSS simulator, LiDAR/IMU simulation, and IAP odometry feedback (PX4 SITL: future integration)

Stage 2 scope (not covered in this report)

- × Continuous-time B-spline joint MAP optimization
- × Explicit active perception (FIM **log det** cost directly in planning)
- × Modifying the SLAM factor graph to absorb planning costs

1.4 Contribution Positioning: U1–U4 Taxonomy

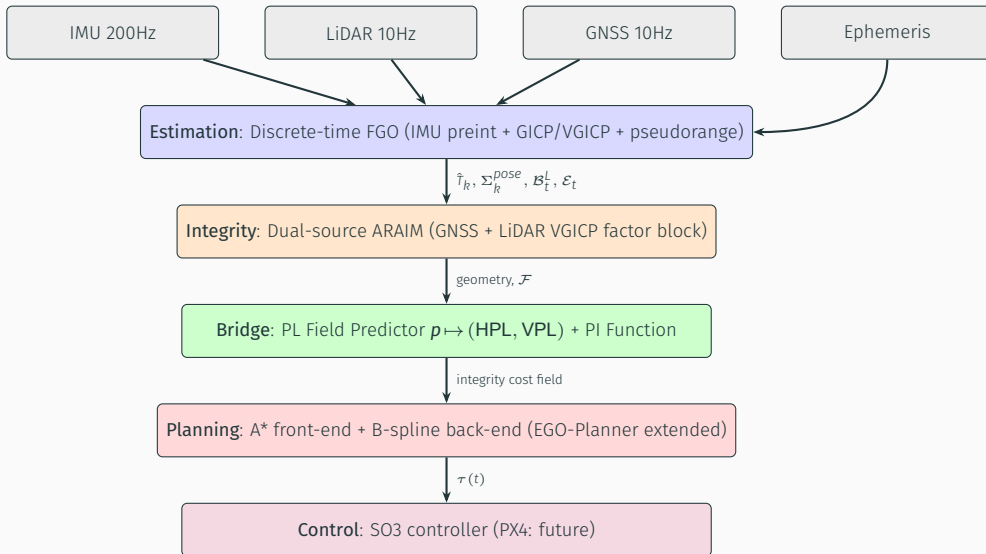
Level	Meaning	This work
U1 joint-opt	SLAM factor graph and planning cost jointly optimized	× (Stage 2)
U2 tightly-coupled	Planning cost feeds back to modify SLAM structure	× (Stage 2)
U3 implicit active	Planning cost senses future information and guides the trajectory	✓(weak form)
U4 metric-unified	SLAM and planner share the same metric (PL)	✓(strong form)

One-sentence positioning

This work implements **strong-form U4 + weak-form U3**: Using PL as the shared metric, it implicitly guides the UAV through high-integrity regions while retaining the engineering and certification advantages of estimation–planning decoupling.

Ch2 Overall Architecture and Design Principles

2.1 Overall Data Flow (End-to-End Architecture)



2.2 Three Core Design Principles

Principle 1: Estimation–Planning Decoupling + Metric Unification

SLAM and the planner are **not** jointly optimized, but they **share** the PL metric. Corresponds to strong-form U4. Clear certification/debugging advantages.

Principle 2: Strict Separation Between Certified Monitoring and Advisory Prediction

- ARAIM = **true** integrity decision at the current pose (designed to be certifiable under WG-C / DO-229 logic)
- PL Predictor = *advisory* integrity prediction at candidate positions (for planning)
- Shared statistical models, but decoupled in function and code path

Principle 3: Local Rolling PL Cache (Off by Default, Demo-Enabled)

Code default: `use_grid=false`, resolution 1 m, box $30 \times 30 \times 8$ m, max-age 2.0 s. In demo11, the grid is enabled via `phase2_use_pl_grid=true`. The planner queries $\widehat{PL}(p)$ by trilinear interpolation and falls back to 0 (advisory cost off) if the field is stale, missing, or out of range.

2.3 Inter-Module Data Contracts (Interface Contracts)

From	To	Payload (actual)	Rate	Path
Sensors	SLAM	raw IMU/LiDAR/GNSS	10–200 Hz	certified
SLAM	ARAIM	$\hat{T}_t, \Sigma_{t,6 \times 6}^{pose}, \mathcal{B}_t^L$, GNSS epoch	10 Hz	certified
SLAM+ARAIM	PL Predictor	visible sats, $\Sigma_{t,6 \times 6}^{pose}, \mathcal{B}_t^L$ summary	2–10 Hz	advisory
PL Predictor	PI Module	local field (HPL, VPL)(p)	2 Hz	advisory
PI Module	Planner	/iap/integrity_front_cost_field	2 Hz	advisory
PI Module	Planner	/iap/integrity_cost_field (back-end)	2 Hz	advisory
ARAIM	Supervisor	alarm flag $\mathbb{I}[\text{PL}_k > \text{AL}]$	10 Hz	certified
Planner	Controller	B-spline $\tau(t)$	10–50 Hz	–

Three engineering values of the contract design:

- **Replaceable:** any module can be replaced by a black box (e.g., FGO replaced by MSCKF) as long as it satisfies the contract
- **Parallelizable:** the Predictor runs in an independent thread and does not block the planner
- **Certifiable:** certified and advisory paths are physically separated

Ch3 Localization Module: GNSS+LiDAR+IMU Tightly Coupled FGO

3.1 Factor-Graph Structure (Baseline Infrastructure)

State (full estimator state):

$$\mathbf{x}_k = [T_k, \mathbf{v}_k, \mathbf{b}_k^a, \mathbf{b}_k^g, \delta t_k^{clk}, \dot{\delta t}_k^{clk}]$$

where δt_k^{clk} and $\dot{\delta t}_k^{clk}$ are the GNSS receiver clock bias and drift, jointly estimated with pose, velocity, and IMU biases.

MAP objective:

$$\hat{\mathbf{X}} = \arg \min_{\mathbf{X}} \sum_k \left\{ \|r_{k,k+1}^{\text{IMU}}\|_{\Sigma_l}^2 + \sum_{F_{j,\ell} \in \mathcal{F}_L} \|r_{k,j,\ell}^{\text{VGICP}}\|_{R_{j,\ell}}^2 + \sum_{s \in \mathcal{S}_k} \|r_{k,s}^{\text{GNSS}}\|_{\sigma_{\text{URA},s}^2}^2 + \|r_0^{\text{prior}}\|_{\Sigma_0}^2 \right\}$$

LiDAR residual (actual: GICP/VGICP factor block):

$$r_m^{\text{gicp}} = T_k \mathbf{p}_m - \mathbf{q}_m, \quad \Sigma_m^{\text{gicp}} = \Sigma_m^{\text{map}} + R_k \Sigma_m^{\text{scan}} R_k^\top$$

The point-to-plane form is a simplified explanatory case; the current IAP implementation primarily exposes *direct GICP/VGICP matching factor blocks* $F_{j,\ell}$ (target frame j , voxelmap level ℓ) to the integrity layer.

GNSS pseudorange residual:

$$r_{k,s}^{\text{GNSS}} = \rho_{k,s} - \|\mathbf{p}_s^w - \mathbf{p}_k\| - c(\delta t_k^{clk} - \delta t_s) - I_s - T_s$$

Positioning of this chapter

This module is **not the main novelty of this work**. We use a mature architecture (LIO-style + tightly coupled GNSS) as the data source for the downstream integrity layer. The novelty starts from Ch4.

3.2 Downstream Data Contract (Snapshot $\mathcal{S}_t$)

Every 100 ms, the estimator publishes one snapshot to the Integrity Layer:

$$\mathcal{S}_t = \{\hat{T}_t, \Sigma_t^p, \Sigma_{t,6 \times 6}^{pose}, \mathcal{E}_t, \mathcal{B}_t^L, \mathcal{T}_t\}$$

Symbol	Meaning
$\hat{T}_t$	current MAP pose estimate (also <code>frame.sigma_p</code> fallback)
Σ_t^p	3×3 position covariance (always available)
$\Sigma_{t,6 \times 6}^{pose}$	6×6 pose covariance, optional via <code>FGOPositionInfo</code> or <code>LidarAraimSnapshot</code>
$\mathcal{E}_t$	visible-satellite epoch $\{s, p_s^w, \sigma_{URA,s}, el_s, az_s\}$
$\mathcal{B}_t^L$	VGICP block snapshot list $\{F_{j,\ell}, \text{RMSE}, \text{inlier frac}, \kappa, \text{age}\}$
$\mathcal{T}_t$	trunk / TDOP detection summary used by the monitor

Note on the information matrix: the integrity layer does **not** consume a full 15×15 Hessian. The LiDAR ARAIM internally reconstructs Λ_0 from the 6×6 pose covariance block, which is already sufficient for the snapshot-downdate hypothesis test (Ch 4.5). The estimator may maintain additional GNSS clock states internally; only the pose / position covariance block is exposed to the advisory path.

Ch4 Dual-Source ARAIM (Core Contribution 1)

4.1 Standard ARAIM Recap (Blanch et al. MHSS)

Fault-hypothesis set:

$$\mathcal{H} = \{H_0\} \cup \{H_s : s \in \mathcal{S}\} \cup \{H_c : c \in \mathcal{C}\}$$

Subset solution under each hypothesis: $\hat{\mathbf{p}}_f = (G_f^\top W_f G_f)^{-1} G_f^\top W_f \mathbf{y}_f$, with covariance $\sigma_{q,f}^2 = [(G_f^\top W_f G_f)^{-1}]_{qq}$.

Solution Separation Test: $|\hat{\mathbf{p}}_{q,0} - \hat{\mathbf{p}}_{q,f}| \leq T_{q,f}$, otherwise raise an alarm.

Protection-level family used in this work (consistent with Ch 4.5 and the actual code):

$$\text{PL}_{q,0} = K_{ff} \sigma_{q,0}$$

$$\text{PL}_{q,f} = |d_{q,f}| + K_{fa,f} \sigma_{ss,q,f} + K_{md,f} \sigma_{q,f} + b_{q,f}, \quad \text{PL}_q = \max(\text{PL}_{q,0}, \max_{f \neq 0} \text{PL}_{q,f})$$

where $d_{q,f}$ is the observed solution separation, $\sigma_{ss,q,f}$ is the separation standard deviation, $\sigma_{q,f}$ is the subset variance, and $b_{q,f}$ is the bias overbound for hypothesis f .

Integrity-risk equation (risk-budget allocation):

$$\text{IR} = P(H_0) P_{md|H_0} + \sum_{f \neq 0} P_{\text{fault},f} P_{md|H_f} \leq \text{IR}_{\text{req}}$$

4.2 GNSS ARAIM Implementation in This System

Configurable fault library. The current implementation uses configurable per-satellite and per-constellation fault priors. The WG-C-style values below are used as *reference defaults*; they must be verified/calibrated before any certification-level claim.

Fault type	Default P_{fault}	Note
Single-satellite (GPS/Galileo)	10^{-5}	Reference default
Single-satellite (GLONASS/BDS)	10^{-4}	More conservative reference
Constellation (per system)	10^{-4}	Reference default

URA model (elevation-dependent):

$$\sigma_{URA,s}^2(el_s) = \sigma_{URE}^2 + \sigma_{tropo}^2(el_s) + \sigma_{iono}^2(el_s) + \sigma_{mp}^2(el_s)$$

with empirical multipath model $\sigma_{mp}(el_s) = a + b e^{-el_s/c}$.

Bias overbound (target model): $b_s = B_{nom} + B_{max,obs}$, where B_{nom} is taken from WG-C as a reference default, and $B_{max,obs}$ is the design target derived from innovation-tail statistics. *Strict per-constellation calibration of these terms is treated as future work.*

4.3 LiDAR ARAIM: From Black-Box ICP to Factor-Block Integrity

Key distinction from conventional LiDAR integrity monitoring

IAP does **not** treat LiDAR ICP as a black-box pose measurement. Instead, the multi-scan VGICP matching cost is inserted into the FGO backend as direct factor blocks. Therefore, the LiDAR fault atom should follow the **internal factor-block structure** rather than individual points or a post-ICP pose factor.

Conventional ICP-as-pose-factor abstraction

$$r_{\text{ICP}} = \log \left(\hat{T}_{\text{ICP}}^{-1} T_k \right), \quad \Delta \Lambda_{\text{ICP}} = J_{\text{ICP}}^{\top} R_{\text{ICP}}^{-1} J_{\text{ICP}}$$

This abstraction hides which scan, keyframe, or resolution level caused the risk.

IAP direct multi-scan VGICP factor abstraction

$$F_{j,\ell} = \text{VGICPMatchingFactor}(\text{source} = \mathcal{S}_t, \text{target} = j, \text{level} = \ell), \quad \mathcal{F}_L = \{F_{j,\ell} \mid j \in \mathcal{T}_t, \ell \in \mathcal{L}\}$$

Consequence

The natural LiDAR ARAIM unit is not a single point, not a single plane, and not an ICP pose factor. It is a **VGICP factor block** indexed by **(source, target, level)**.

4.4 IAP LiDAR Fault-Hypothesis Library

LiDAR fault library induced by the VGICP block structure

$$\mathcal{H}_L = \{H_0, H_{\text{source}}\} \cup \{H_{\text{target}}(j) : j \in \mathcal{T}_t\} \cup \{H_{\text{level}}(\ell) : \ell \in \mathcal{L}\}.$$

Fault mode	Meaning	Removed VGICP blocks
H_{source}	Current deskewed source scan is unreliable	$\mathcal{B}(H_{\text{source}}) = \{F_{j,\ell}, \forall j, \ell\}$
$H_{\text{target}}(j)$	Target frame / keyframe j is inconsistent	$\mathcal{B}(H_{\text{target}}(j)) = \{F_{j,\ell}, \forall \ell\}$
$H_{\text{level}}(\ell)$	Voxelmap resolution level ℓ is unreliable	$\mathcal{B}(H_{\text{level}}(\ell)) = \{F_{j,\ell}, \forall j\}$

Interpretation

- H_{source} : dynamic contamination, skew failure, poor current scan quality.
- $H_{\text{target}}(j)$: wrong historical keyframe pose, corrupted target map, large viewpoint inconsistency.
- $H_{\text{level}}(\ell)$: coarse level gives biased geometry, fine level becomes noisy or unstable.

Innovation

The LiDAR fault tree is not manually invented. It is induced by the actual IAP VGICP factor organization: **source** $\times$ **target** $\times$ **level**.

4.5 Snapshot Downdate: Real-Time LiDAR Subset ARAIM

Linearized contribution of one VGICP block

$$(J_{j,\ell}, r_{j,\ell}, R_{j,\ell}) = \text{linearize}(F_{j,\ell}, \hat{x}_0)$$

$$\Delta\Lambda_{j,\ell} = J_{j,\ell}^\top R_{j,\ell}^{-1} J_{j,\ell}, \quad \Delta g_{j,\ell} = J_{j,\ell}^\top R_{j,\ell}^{-1} r_{j,\ell}$$

Subset system under LiDAR fault hypothesis H_f

$$\Lambda_f = \Lambda_0 - \sum_{F_{j,\ell} \in \mathcal{B}(H_f)} \Delta\Lambda_{j,\ell}, \quad g_f = g_0 - \sum_{F_{j,\ell} \in \mathcal{B}(H_f)} \Delta g_{j,\ell}$$

$$\delta x_f = \Lambda_f^{-1} g_f, \quad \hat{x}_f = \hat{x}_0 \boxplus \delta x_f, \quad \Sigma_f = \Lambda_f^{-1}$$

Solution separation and LiDAR protection level

$$d_f = \hat{x}_0 \boxminus \hat{x}_f, \quad \sigma_{q,f} = \sqrt{e_q^\top \Sigma_f e_q}$$

$$T_{q,f} = K_{fa,f} \sigma_{ss,q,f}, \quad \text{PL}_{q,f}^L = |e_q^\top d_f| + T_{q,f} + K_{md,f} \sigma_{q,f} + b_{q,f}$$

$$\text{PL}_q^L = \max_{f \in \mathcal{H}_L} \text{PL}_{q,f}^L$$

Real-time design decision

We do **not** rerun the nonlinear smoother for every LiDAR hypothesis. We use the current linearized FGO snapshot and perform information-matrix downdate, which is compatible with online ARAIM timing.

4.6 ICP/VGICP Quality as Integrity Evidence (Current vs Future)

Current implementation (`block_risk_score()`): a four-term normalized weighted sum

$$\gamma_{j,\ell} = w_r \frac{\text{RMSE}_{j,\ell}}{\text{RMSE}_{ref}} + w_i (1 - \text{InlierFrac}_{j,\ell}) + w_c \log \kappa_{j,\ell} + w_a \frac{\text{Age}_j}{\text{Age}_{ref}}$$

- $\text{RMSE}_{j,\ell}$: VGICP residual RMS (`rmse_proxy`)
- $\text{InlierFrac}_{j,\ell}$: inlier ratio of the block
- $\kappa_{j,\ell}$: condition-number / degeneracy proxy (`cond_proxy`)
- Age_j : temporal/topological age of target keyframe (`age_sec`)

Current fault-prior model (mode-level, configurable):

$$P_{\text{fault}}(H_{\text{source}}) = p_{\text{source}}, \quad P_{\text{fault}}(H_{\text{target}}(j)) = p_{\text{target}}, \quad P_{\text{fault}}(H_{\text{level}}(\ell)) = p_{\text{level}}$$

$\gamma_{j,\ell}$ updates a `gamma_mode` indicator that scales the bias overbound, not P_{fault} .

Bias overbound (current):

$$b_{q,H_{\text{target}}(j)} = \alpha_q \max_{\ell} \gamma_{j,\ell}, \quad b_{q,H_{\text{level}}(\ell)} = \alpha_q \max_j \gamma_{j,\ell}, \quad b_{q,H_{\text{source}}} = \alpha_q \max_{j,\ell} \gamma_{j,\ell}$$

Future extension / calibratable model

A learned per-block prior using deskew error, dynamic-point ratio, and normal-distribution symmetry:

$$P_{\text{fault},j,\ell} = \text{clip}(\sigma(a_0 + a_1 \gamma + a_2 \text{Dyn} + a_3 \text{Sym}))$$

`DeskewErr`, `Dyn`, `Sym` and the sigmoid mapping are *not* in the current `block_risk_score()` / `enumerate_hypotheses()`; they are Stage-2 calibration extensions.

4.7 Dual-Source Fusion Strategy (Conservative Fusion)

After computing the two PL channels independently, the runtime monitor performs a per-axis conservative max over $\text{PL}_{E/N/U}$, then updates HPL/VPL from the dominating source:

$$\text{PL}_q^{\text{fused}} = \max(\text{PL}_q^{\text{GNSS}}, \text{PL}_q^{\text{LiDAR}}).$$

Why conservative max instead of a joint fault tree?

- **Conservatism:** the fused PL never underestimates risk from either channel
- **Decoupling:** GNSS and LiDAR fault libraries can evolve independently
- **Real-time:** complexity is additive, not combinatorial
- **Certifiability path:** minimal modification to the certified channel

Risk-budget interpretation (rationale, *not* an implemented metric)

At a paper level, the fused-risk upper bound is

$$\text{IR}^{\text{fused}} \leq \text{IR}^{\text{GNSS}} + \text{IR}^{\text{LiDAR}} - \text{IR}^{\text{GNSS}} \text{IR}^{\text{LiDAR}} \approx \text{IR}^{\text{GNSS}} + \text{IR}^{\text{LiDAR}}.$$

This is a theoretical upper-bound rationale for the conservative max. The runtime monitor fuses PL/HPL/VPL and the dominating source label, but does **not** emit an explicit fused-IR scalar.

Complexity: a joint sensor-fault tree scales as $\mathcal{O}(2^{|\mathcal{S}|+|\mathcal{B}|})$; conservative max scales as $\mathcal{O}(|\mathcal{S}| + |\mathcal{B}|)$, suitable for online UAV planning.

4.8 Positioning Relative to Mainstream Work

Method	Fault enumeration	Multi-sensor	Real-time	Feeds planner
Blanch 2015 (MHSS)	GNSS satellite / constellation	×	✓	×
Joerger & Pervan 2019	Sensor + satellite	✓	partial	×
LiDAR RAIM / map RAIM	Landmark / map feature	partial	✓	×
This work	GNSS + IAP VGICP block	✓	✓	✓

Ch4 Contributions

1. **Factor-aware LiDAR ARAIM**: extend ARAIM from satellite faults to IAP VGICP factor-block faults.
2. **Structured LiDAR fault atoms**: source scan, target keyframe, voxelmap resolution level.
3. **Snapshot downdate**: avoid rerunning the nonlinear FGO for every LiDAR hypothesis.
4. **ICP-quality-driven bias overbound**: residual tail, inlier ratio, degeneracy, and target age.
5. **Conservative GNSS–LiDAR fusion**: real-time feasibility, no combinatorial joint fault tree.

Ch5 PL Field Predictor (Core Contribution 2)

5.1 Motivation: Why Is ARAIM Not Enough?

Three mismatches:

1. **Temporal mismatch:** ARAIM gives $\mathbf{PL}(\mathbf{p}_t, t)$, the planner needs $\mathbf{PL}(\mathbf{p}_{t+\Delta}, t + \Delta)$, $\Delta \in [0.5, 5]$ s.
2. **Spatial mismatch:** ARAIM is a scalar at the current point, while the planner needs a **field** $\mathbf{PL} : \mathbb{R}^3 \rightarrow \mathbb{R}_+$.
3. **Computational infeasibility:** embedding full MHSS into every B-spline gradient evaluation $\Rightarrow$ one replan $\approx n_{cp} \cdot n_{iter} \cdot T_{MHSS} = 20 \cdot 50 \cdot 5 \text{ ms} = 5 \text{ s}$, infeasible.

Design conclusion: ARAIM online computation and planner-side queries must be **decoupled**. $\Rightarrow$ a precomputed local scalar field + fast interpolation query interface.

Role of the PL Field Predictor

A **lightweight surrogate** of ARAIM in **space and future time**, used **only** on the advisory path.

5.2 Formal Module Definition

Domain (default code parameters): a local box centered at the current UAV position p_t^0 :

$$\Omega_t = p_t^0 + [-L_x/2, L_x/2] \times [-L_y/2, L_y/2] \times [-L_z/2, L_z/2]$$

Code defaults: $L_x = L_y = 30$ m, $L_z = 8$ m, resolution 1 m, max-age 2.0 s, `use_grid=false`. Demo11 launch overrides these via `phase2_use_pl_grid=true` and `phase2_pl_model=gnss_geometry_araim`.

Range and mapping:

$$\mathcal{P}_t : \Omega_t \rightarrow \mathbb{R}_{\geq 0}^2, \quad p \mapsto (\text{HPL}(p), \text{VPL}(p))$$

Decomposition into five serial submodules:

$$p \xrightarrow{A} v_s(p) \xrightarrow{B} \Delta\Lambda_L(p) \xrightarrow{C} \Lambda_{pred}(p) \xrightarrow{D} \text{PL}(p) \xrightarrow{E} \text{cached grid}$$

Key properties (Section 5.4):

- Local Lipschitz continuity: $|\text{PL}(p_1) - \text{PL}(p_2)| \leq L_{\text{PL}} \|p_1 - p_2\|$
- Self-consistency: $\widehat{\text{PL}}(p_t^0) \approx \text{PL}_{\text{ARAIM}}(p_t^0)$

5.3 Submodule A: Satellite Geometry Predictor

Function: given a candidate position $\mathbf{p}$, estimate the visible-satellite set and geometry at future time $t + \Delta$.

Visibility indicator:

$$v_s(\mathbf{p}, t + \Delta) = \mathbb{K}[el_s(\mathbf{p}, t + \Delta) > el_{min}] \cdot \mathbb{K}[\text{LoS}(\mathbf{p}, \mathbf{p}_s^W(t + \Delta)) \text{ unblocked}]$$

Elevation computation (satellite positions extrapolated from $\mathcal{E}_t$):

$$el_s(\mathbf{p}, t + \Delta) = \arcsin\left(\frac{(\mathbf{p}_s^W(t + \Delta) - \mathbf{p})^\top \mathbf{u}_{up}(\mathbf{p})}{\|\mathbf{p}_s^W(t + \Delta) - \mathbf{p}\|}\right)$$

LoS blockage test (optional): raycast in the occupancy map; in map-free scenes set $\text{LoS} = 1$.

Geometry matrix:

$$G(\mathbf{p}) = \begin{bmatrix} -\mathbf{e}_{s_1}^\top(\mathbf{p}) & 1 \\ \vdots & \vdots \\ -\mathbf{e}_{s_N}^\top(\mathbf{p}) & 1 \end{bmatrix}, \quad \mathbf{e}_s(\mathbf{p}) = \frac{\mathbf{p}_s^W - \mathbf{p}}{\|\mathbf{p}_s^W - \mathbf{p}\|}$$

Retain only rows with $v_s(\mathbf{p}) = 1$.

5.3 Submodule B: LiDAR Information Predictor (Optional)

Function: estimate the LiDAR information gain at candidate position p .

Code default: `use_lidar_observability=false`, i.e. this submodule is *off by default*. When enabled, it acts as a confidence modulator on the current LiDAR information block.

Lightweight surrogate (α_L modulator):

$$\Delta\Lambda_{\text{LiDAR}}(p) = \alpha_L(p) \cdot \Delta\Lambda_{\text{LiDAR}}^{(t_0)}$$

where

- $\Delta\Lambda_{\text{LiDAR}}^{(t_0)}$ is the actual LiDAR information contribution at the current time
- $\alpha_L(p) \in [0, 1]$ is a **confidence modulator**, not a fault probability

Modulator design:

$$\alpha_L(p) = \exp(-\|p - p_t^0\|^2 / d_L^2) \cdot \mathbb{K}[\text{within sensor range}]$$

Engineering rationale: future positions have no source scan; running full VGICP block ARAIM at every grid voxel is over-precise for the advisory layer. The confidence modulator captures the dominant range-decay behavior at near-zero cost.

5.3 Submodule C: Fused FIM Assembler (Design Decision 2)

Basic assembly:

$$\Lambda_{pred}^{(0)}(\mathbf{p}) = \Lambda_{base} + \Delta\Lambda_{GNSS}(\mathbf{p}) + \alpha_L(\mathbf{p}) \Delta\Lambda_{LiDAR}^{(t_0)}$$

GNSS information contribution (position-dependent):

$$\Delta\Lambda_{GNSS}(\mathbf{p}) = G^\top(\mathbf{p}) W(\mathbf{p}) G(\mathbf{p}), \quad W(\mathbf{p}) = \text{diag}(1/\sigma_{URA,s}^2(el_s(\mathbf{p})))$$

Position-dependent lightweight covariance growth (current code)

The current `PredictedIntegrityComputer::sigma_grow_at()` only depends on visible-satellite deficit and mean canopy density:

$$\sigma_{grow}(\mathbf{p}) = \sigma_0 \left[1 + \beta_v \frac{N_{nom} - N_{vis}(\mathbf{p})}{N_{nom}} + \beta_\kappa \bar{\kappa}(\mathbf{p}) \right]$$

- Term 1: visible-satellite-deficit penalty
- Term 2: mean canopy / GDOP-degeneracy proxy ($\bar{\kappa}$)
- Optional extension (not enabled in current code): a distance-anchor term $\beta_d \|\mathbf{p} - \mathbf{p}_t^0\|$ to penalize farther-from-anchor queries; reserved as a Stage 2 calibration knob.

5.3 Submodule D: GNSS Geometry-Only ARAIM PL + Optional Fused-FIM PL

Current implementation (**PredictedAraimComputer::predict_geometry()**): *geometry-only* mode. The Predictor predicts visible satellites from a candidate position, builds the geometry matrix $G(\mathbf{p})$, and calls a zero-residual ARAIM evaluator (no full per-grid MHSS over LiDAR hypotheses).

Compute fault-free protection level (subset variance from $\Sigma_{pred}(\mathbf{p})$):

$$\sigma_{q,0}^2(\mathbf{p}) = [\Sigma_{pred}(\mathbf{p})]_{qq}, \quad \text{PL}_{q,0}(\mathbf{p}) = K_{ff} \sigma_{q,0}(\mathbf{p}) + b_{q,0}(\mathbf{p})$$

For GNSS satellite hypotheses $f \in \mathcal{H}_{\text{GNSS}}$ (with downdate $\Lambda_{pred} - \Delta\Lambda_f$):

$$\text{PL}_{q,f}(\mathbf{p}) = K_{fa,f} \sigma_{ss,q,f}(\mathbf{p}) + K_{md,f} \sigma_{q,f}(\mathbf{p}) + b_{q,f}(\mathbf{p})$$

HPL / VPL:

$$\text{HPL}(\mathbf{p}) = \max\left(\text{PL}_{h,0}(\mathbf{p}), \max_{f \in \mathcal{H}_{\text{GNSS}}} \text{PL}_{h,f}(\mathbf{p})\right)$$

analogous form for VPL.

Optional conservative fused-FIM PL (when `use_fused_fim_grid=true`): combine GNSS base information with the LiDAR-observability FIM, compute $\text{HPL}^{\text{fused}}$, and take per-axis max with the GNSS-only HPL.

5.3 Submodule E: Field Cache & Interpolator

Grid storage (code defaults):

- Resolution $\Delta = 1$ m
- Size $30 \times 30 \times 8 = 7200$ voxels
- Each voxel stores $(\text{HPL}, \text{VPL}) \in \mathbb{R}^2$ plus risk-band metadata
- Total memory ≈ 60 kB

Query interface (trilinear interpolation):

$$\widehat{\text{PL}}(p) = \sum_{i=0}^7 w_i(p) \text{PL}(v_i), \quad \sum_i w_i = 1$$

Gradient (when needed by the back-end optimizer):

$$\nabla_p \widehat{\text{PL}}(p) = \sum_{i=0}^7 [\nabla_p w_i(p)] \text{PL}(v_i)$$

Update strategy:

- Eager precompute around the UAV at 2 Hz
- Lazy on-demand for first-touched voxels
- **Field aging**: voxels older than `grid_max_age_s=2.0s` are *stale*; the planner falls back to 0 advisory cost

5.4 Summary of Key Design Decisions (Engineering Contributions)

Decision	Full-FIM route	Route in this work
LiDAR future contribution	Virtual VGICP block ARAIM at every voxel	Optional $\alpha_L(p)$ confidence modulator (off by default)
Covariance forward propagation	Full Hessian propagation along trajectory	Position-dependent σ_{grow} (visible-deficit + canopy)
ARAIM-Predictor coupling	Embedded shared computation graph	Functional decoupling, shared statistical model
Online subset enumeration	Full MHSS at every voxel	Geometry-only ARAIM at voxels; full MHSS only at the current pose

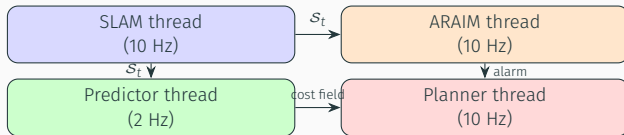
Proposition 1 (self-consistency target)

At $p = p_t^0$, with proper calibration of β_v, β_κ , the Predictor satisfies

$$|\widehat{\text{PL}}(p_t^0) - \text{PL}_{\text{ARAIM}}(p_t^0)| \leq \epsilon_0$$

with $\epsilon_0 \rightarrow 0$ when $\sigma_{grow}(p_t^0) \rightarrow 0$. A formal closed-form conservatism bound is reserved as future work; numerical Monte-Carlo verification is used as the substitute (Ch 8.4).

5.5 Real-Time Design and Overall Timing



Key timing constraints (with default $30 \times 30 \times 8$ box):

- One full Predictor scan over 7200 voxels: ≤ 200 ms on Intel i7-class CPU
- Planner replan period: 100 ms; each replan queries ~ 200 points $\Rightarrow < 5$ ms
- Field aging threshold: 2.0 s (`grid_max_age_s`)

Degradation strategy (Graceful Degradation):

$$c_{PI}(p) = \begin{cases} \pi(\text{HPL}(p), \text{VPL}(p)), & \text{field fresh} \wedge p \in \Omega_t \\ 0, & \text{field stale} \vee p \notin \Omega_t \end{cases}$$

$\Rightarrow$ If the Predictor fails or is disabled, the planner degrades to standard EGO-Planner without introducing additional risk.

Ch6 PI Function and Two-Stage Cost Architecture (Core Contribution 3)

6.1 Role of the PI Function

Definition: a mapping from integrity metrics to planning cost

$$\pi : \mathbb{R}_{\geq 0}^2 \rightarrow \mathbb{R}_{\geq 0}, \quad (\text{HPL}, \text{VPL}) \mapsto c_{PI}$$

Three invocation points in the system:

1. **Front-end A* edge cost** via `/iap/integrity_front_cost_field`
2. **Back-end planner cost** via `/iap/integrity_cost_field`
3. Safety Supervisor: $\nVdash[\text{PL}_k^{\text{ARAIM}} > \text{AL}]$ triggers a safety action

Key Design Philosophy

The PI function uses **different forms** in the front-end and back-end:

- Front end: π_{front} is based on a **dimensionless ratio** and used as a *multiplicative edge inflation*
- Back end: π_{back} is based on a **dimensional hinge** on AL margin

Together, they form a **two-stage cost architecture**—an architectural contribution of this work.

6.2 Back End: B-spline Hinge-based PI Cost

B-spline trajectory: $\tau(t) = \sum_i B_i(t) \mathbf{c}_i$, control-point set $\{\mathbf{c}_i\}$.

PI cost (hinge squared):

$$J_{PI}^{back}(\tau) = \sum_i \left\{ w_H [\max(0, \text{HPL}(\mathbf{c}_i) + m_H - \text{HAL})]^2 + w_V [\max(0, \text{VPL}(\mathbf{c}_i) + m_V - \text{VAL})]^2 \right\}$$

Implementation status (*partial*)

The hinge cost form is implemented in `PICostAdapter`. In the current pipeline, the planner consumes *precomputed cost (and gradient when available) samples* from `/iap/integrity_cost_field`, rather than back-propagating $\nabla \text{HPL}(\mathbf{c}_i)$ analytically inside the optimizer. Fully analytic PL-gradient back-propagation through the predictor is kept as a method-level extension.

Two improvements over standard π_1 :

1. **Dynamic AL**: use mission-level **HAL**, **VAL** (isomorphic to ARAIM)
2. **Explicit margin** m_H, m_V : build a buffer before the AL boundary

Typical values: **HAL** = 10 m, **VAL** = 6 m, $m_H = 1.5$ m, $m_V = 1.0$ m, $w_H = w_V = 2.0$.

6.3 Front End: A* Ratio-based PI Cost as Multiplicative Edge Inflation

Define a dimensionless integrity occupancy ratio:

$$r(p) = \max\left(\frac{\text{HPL}(p)}{\text{HAL}}, \frac{\text{VPL}(p)}{\text{VAL}}\right) \in [0, \infty)$$

Piecewise quadratic cost (risk band):

$$c_{pl}^{front}(p) = \begin{cases} 0, & r \leq 0.7 \quad (\text{SAFE}) \\ \left(\frac{r - 0.7}{0.3}\right)^2, & 0.7 < r \leq 1 \quad (\text{CAUTION}) \\ 1 + (r - 1)^2, & r > 1 \quad (\text{ALARM}) \end{cases}$$

A* edge cost (multiplicative inflation, current implementation):

$$e(p_i, p_j) = \|p_i - p_j\| \cdot (1 + \lambda_{front} c_{pl}^{front}(p_j))$$

where $c_{pl}^{front}(p_j)$ is queried by nearest-neighbor lookup on the IntegrityCostMap; if the field is stale, missing, or out of radius, $c_{pl}^{front} = 0$ and the edge degenerates to standard EGO geometry.

Why a ratio (and why multiplicative)?

- **Dimensionless:** ratio is independent of HAL/VAL absolute calibration
- **Heuristic-friendly:** multiplicative inflation preserves admissibility of the Euclidean heuristic
- **Zero interference in SAFE regions:** when $r \leq 0.7$, behavior reduces to original EGO
- **Strong topology discrimination:** penalizes already in CAUTION, encouraging high-integrity corridors

6.4 Engineering Value of the Two-Stage Architecture

Stage	Cost form	Complexity	Role
Front-end A*	Ratio, multiplicative edge inflation	$\mathcal{O}(1)$ per edge	Global topology guidance: avoid high-PL corridors
Back-end spline	B-Hinge squared, dimensional	$\mathcal{O}(n_{cp} \cdot n_{iter})$	Local refinement: build margin at the AL boundary

Corresponding classic motion-planning paradigm:

front-end heuristic search + back-end local optimization

Compatible with mainstream architectures such as Fast-Planner and EGO-Planner.

Why not use a single cost everywhere?

- Hinge only: front end may pick an average-high-PL corridor with every point still below HAL, which back end cannot recover
- Ratio only: dimensionless back-end cost conflicts with J_{smooth} and $J_{collision}$ scales

⇒ Separation of responsibilities is architecturally necessary.

6.5 Graceful Degradation and Safety Supervisor

Field health monitoring:

$$\text{health}(t) = \mathbb{I}[t - t_{\text{last update}} < \text{grid_max_age_s}] \cdot \frac{|\text{fresh voxels}|}{|\Omega_t|}$$

Automatic degradation strategy:

$$c_{PI}^{\text{effective}}(p) = \text{health}(t) \cdot c_{PI}(p)$$

Safety Supervisor (engineering state machine, **IntegrityMonitor**):

$$\text{State}(t) = \begin{cases} \text{UNSAFE}, & \text{PL}_k \geq \text{AL} \\ \text{SAFE_EXCLUDED}, & \text{fault detected} \wedge \text{PL}_k < \text{AL} \\ \text{SAFE}, & \text{recovered after consecutive safe frames} \end{cases}$$

When unsafe, the system executes one of:

- Decelerate/hover and recompute the front-end A* path
- Roll back to the trajectory from the most recent fresh field
- Trigger emergency landing

Ch7 Planning Module: EGO-Planner Extension

7.1 EGO-Planner Baseline Recap

Original objective function (Zhou et al. 2021):

$$J_{EGO} = \lambda_s J_{smooth} + \lambda_c J_{collision} + \lambda_d J_{dynamic}$$

Smooth cost: $J_{smooth} = \sum_i \|\mathbf{c}_i - 2\mathbf{c}_{i+1} + \mathbf{c}_{i+2}\|^2$

Collision cost (ESDF-based): $J_{collision} = \sum_i [\max(0, d_{thr} - d_{ESDF}(\mathbf{c}_i))]^2$

Why choose EGO-Planner?

- Decoupled architecture matches the discrete-time SLAM route in this work
- B-spline control points are the most natural injection point for PI cost
- ZJU-FAST-Lab open-source implementation is mature
- ESDF collision cost and PL hinge cost have isomorphic mathematical forms $\Rightarrow$ gradient templates can be reused

7.2 Optimization Problem After Integrity Extension

Extended back-end objective:

$$J_{total} = \lambda_s J_{smooth} + \lambda_c J_{collision} + \lambda_d J_{dynamic} + \lambda_{\pi}^{back} J_{PI}^{back}$$

Front-end A* extension (multiplicative edge cost, matches IAP code):

$$e(n, n') = \|p_n - p_{n'}\| \cdot (1 + \lambda_{front} c_{PI}^{front}(p_{n'})) + \lambda_o c_{occ}(n')$$

The Euclidean heuristic $h(n)$ is unchanged (admissibility preserved when $\lambda_{front} \geq 0$).

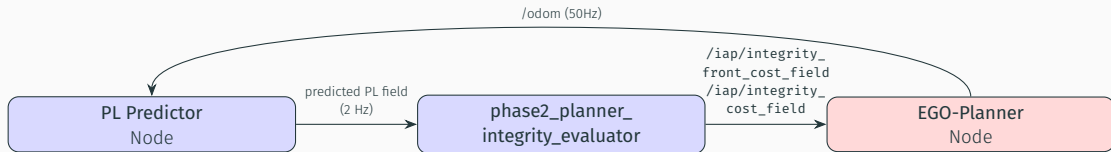
Typical hyperparameter values:

Parameter	Value	Parameter	Value
λ_s	10.0	λ_{front}	5.0
λ_c	10000.0	λ_{π}^{back}	2.0
λ_d	0.01	B-spline order	3

Implementation note: the back-end PI cost is formulated as a hinge loss; in the current implementation, the planner consumes *precomputed cost/gradient samples* from the integrity cost field rather than back-propagating ∇ HPL analytically through the predictor. Fully analytic

7.3 Cross-Process IntegrityCostMap Communication Design

Engineering choice: do not fork the EGO-Planner source; communicate via ROS2 topics.



Two cost fields with distinct schemas:

- Front-end A* field `/iap/integrity_front_cost_field` (PointCloud2):

`{x, y, z, HPL, VPL, HAL, VAL, cost, risk_band_code}`

consumed by EGO-Planner front-end A* via the multiplicative edge inflation in §7.2.

- Back-end / evaluator field `/iap/integrity_cost_field` (PointCloud2): `{x, y, z, cost, gradx, grady, gradz, risk_band}` when gradient samples are available.

Advantages: no modification of EGO-Planner memory model; native RViz visualization (publish = debug); multiple subscribers (Supervisor also listens).

Ch8 Experimental Validation

8.1 Simulation Platform and demo11 Scenario

Platform (current main loop): [ROS2-based SO3 UAV simulation](#) on Ubuntu / ROS2 Humble (Intel i7-class CPU). Key chain: `traj_server` → `SO3 controller` → `SO3 plant` → `IAP odom feedback`, with EGO-Planner, GNSS simulator, LiDAR/IMU simulation.

PX4 SITL / Gazebo: planned future integration, not the current validation platform.

demo11 “Stratified Canopy” scenario design:

- Corridor length ≈ 100 m; lower-canopy density $\rho_{low} = 0.1$, upper-canopy density $\rho_{up} = 0.6$
- Left-right asymmetry: dense canopy on one side (heavy GNSS blockage), sparse on the other
- Expected behavior: UAV chooses the sparse-canopy corridor (better PL) over the geometric shortest path

Run mode: `use_so3_dynamics:=true`; demo11 enables `phase2_use_pl_grid=true`, `phase2_pl_model=gnss_geometry_araim`.

Comparison configurations (via launch flags):

- *Baseline:* `planner_use_integrity_cost=false`, `planner_use_integrity_front_search=false`
- *Front-only:* `planner_use_integrity_front_search=true`
- *Full:* above two enabled jointly with global integrity search

8.2 Completed Validation (Preliminary)

Single-module functional smoke tests (✓, qualitative):

- FGO+ARAIM stack runs end-to-end in demo11 without crash
- ARAIM HPL/VPL time-series rises during simulated GNSS blockage (qualitative trend matches expectation)
- PL Field Predictor visualization in RViz shows clear left-right anisotropy in stratified canopy
- PI cost is injected into EGO-Planner via `/iap/integrity_front_cost_field`; closed-loop pipeline runs

End-to-end closed loop (✓, small-scale):

- Several demo11 closed-loop runs completed without runtime errors
- Visual inspection: trajectories under *Full* configuration tend to bias toward the sparser-canopy side, while *Baseline* traces a geometrically shorter line

Status: preliminary smoke tests / qualitative observations

Quantitative numbers (e.g., APE, peak HPL, success counts) are *not yet* statistically validated. Large-scale benchmarking, log archiving, and CSV-based statistics are scheduled in the next two weeks (§8.3, §8.4).

8.3 Baseline Comparison Experiment Design (Planned)

Comparison configurations matching the README launch flags:

ID	Description	Front edge cost	Back-end cost	Status
B1	Baseline (standard EGO)	off	off	primary control
B2	Front-only	ratio (multiplicative)	off	primary contrast
B4	Full system (this work)	ratio (multiplicative)	hinge	primary target
B3	Back-end-only	off	hinge	optional ablation

B1 / B2 / B4 are the three primary configurations directly supported by the README launch flags. B3 (back-end-only) is enabled only if the integrity back-end cost field is configured independently of the front-end injection.

Evaluation metrics:

Metric	Definition	Expected
Success rate	# successful arrivals / # total runs	↑ B4
Avg HPL / Max HPL	$\overline{\text{HPL}}$ and $\max \text{HPL}$ along the trajectory	↓ B4
Trajectory length	$\int \ \dot{\tau}\ dt$	close to B1
Replan compute time	average ms per replan	$B4 \leq B1 + 20\%$
Supervisor trigger rate	# Alarm / mission time	↓ B4

Statistical test: Mann–Whitney U on each metric pair, $\alpha = 0.05$.

8.4 Self-Consistency Sanity Check (Critical)

Purpose: verify that the Predictor matches the certified ARAIM at $\mathbf{p} = \mathbf{p}_t^0$.

Metric:

$$\epsilon_{self}(t) = |\widehat{\text{HPL}}(\mathbf{p}_t^0) - \text{HPL}_t^{\text{ARAIM}}| / \text{HPL}_t^{\text{ARAIM}}$$

Acceptance target:

$$\mathbb{E}[\epsilon_{self}] < 5\%, \quad \text{P95}[\epsilon_{self}] < 15\%$$

(initial loose targets; tightened after σ_{grow} calibration in W3)

Visualizations:

- Scatter plot $\widehat{\text{HPL}}(\mathbf{p}_t^0)$ vs $\text{HPL}_t^{\text{ARAIM}}$ (target: lie on $y = x$)
- Time-series overlay of the two curves
- Residual histogram (target: zero-mean, small variance)

Failure diagnosis

If ϵ_{self} stays above target: $\sigma_{grow}(0)$ is mis-calibrated (should approach 0 at $\mathbf{p} = \mathbf{p}_t^0$), or the constant component of α_L has not been zeroed.

8.5 Scenario-Response Test Matrix (Planned)

Scenario	Key feature	Expected PL-field response
Open field	many satellites visible; sparse LiDAR features	nearly constant, near-isotropic; low HPL
Urban canyon	vertical satellite blockage; lateral visibility	VPL rises in canyon; horizontal anisotropy
Corridor (demo11)	LiDAR longitudinal weakness; GNSS variable	longitudinal HPL rise; lateral stable; strong anisotropy
Tunnel entrance	GNSS visibility drops sharply	steep spatial gradient; CAUTION→ALARM transition
Stratified canopy	sparse/dense asymmetry	lower PL on sparse side $\Rightarrow$ B4 chooses sparse side

Validation method:

1. Collect 2D slices of $\mathbf{PL}(\mathbf{p})$ in each scenario (qualitative match with physical intuition)
2. Quantitative anisotropy ratio $\lambda_{\max}/\lambda_{\min}$ of the spatial gradient tensor
3. Trajectory-shape comparison between B1 and B4

Ch9 Summary and Next Steps

9.1 Core Contribution Checklist

Positioning relative to U1–U4:

- **U4 (metric-unified):** ✓ architecture completed—PL is the shared SLAM-to-planner metric
- **U3 (implicit active perception):** ✓ architecture completed, empirical validation ongoing
- U1 (joint optimization): × Stage 2
- U2 (tightly-coupled): × Stage 2

Three claimable contributions (with honest scope statements):

1. **Factor-aware LiDAR ARAIM:** VGICP factor-block fault library, snapshot downdate, ICP-quality-driven bias overbound; conservative max fusion with GNSS ARAIM
2. **Lightweight PL Field Predictor:** GNSS geometry-only ARAIM at voxels with optional fused-FIM PL, position-dependent σ_{grow} surrogate, decoupled 2 Hz advisory thread
3. **Two-stage PI cost architecture:** front-end ratio-based *multiplicative edge inflation* (topology) + back-end hinge cost (margin)

Underlying engineering contributions:

- Physical separation between certified monitor and advisory predictor

9.2 Completed Work Checklist (Honest Status)

Chapter	Work item	Status
Ch3 SLAM	FGO (IMU+VGICP+GNSS pseudorange) running in demo11	✓
Ch3 SLAM	Snapshot $\mathcal{S}_t$ publishing interface (pose cov + block list)	✓
Ch4 ARAIM	GNSS MHSS implementation + URA model	✓
Ch4 ARAIM	LiDAR source/target/level fault hypotheses	✓
Ch4 ARAIM	LiDAR block risk score and bias overbound	✓
Ch4 ARAIM	Conservative GNSS-LiDAR max fusion	✓
Ch4 ARAIM	Learned per-block $P_{fault,b}$ (sigmoid + Dyn/Sym/DeskewErr)	<i>future work</i>
Ch5 Predictor	GNSS geometry-only ARAIM grid + optional fused-FIM PL	✓
Ch5 Predictor	Trilinear interpolation + field aging + degradation	✓
Ch5 Predictor	σ_{grow} (visible-deficit + canopy) surrogate	✓
Ch5 Predictor	Full subset-MHSS / IR field over all source hypotheses	<i>future work</i>
Ch6 PI	Front-end ratio cost field (multiplicative edge inflation)	✓
Ch6 PI	Back-end hinge cost formulation / cost-field gradient interface	<i>partial</i>
Ch7 Planner	Decoupled EGO-Planner extension via ROS2 topics	✓
Ch8 Exp	demo11 closed-loop smoke tests (qualitative)	✓
Ch8 Exp	Large-scale 3-baseline comparison (B1/B2/B4, ≥ 60 runs)	<i>ongoing</i> ₄₂
Ch8 Exp	Quantitative self-consistency sanity check	<i>ongoing</i>

9.3 Near-Term To-Do List (6-Week Rolling Plan)

Week	Task	Deliverable
W1	demo11 B1/B2/B4 large-scale runs (≥ 60 each), with logs	success-rate / PL / trajectory-length CSVs and figures
W2	Self-consistency sanity check + scenario-response slices	all figures for §8.4 and §8.5
W3	σ_{grow} calibration report + α_L activation experiment	calibration document + before/after curves
W4	Paper draft (Method + Experiment)	draft v0.5
W5	UrbanNav qualitative replay (optional) + revision	draft v0.9
W6	Advisor review + submission (T-IV / ICRA / IROS)	submission

Key risks and mitigation:

- Risk A: closed-form σ_{grow} conservatism proof is intractable $\Rightarrow$ Monte-Carlo numerical validation
- Risk B: Predictor latency exceeds 200 ms with default box $\Rightarrow$ shrink box / lower resolution / reduce hypothesis enumeration
- Risk C: UrbanNav has no ground-truth PL $\Rightarrow$ qualitative-only, excluded from quantitative comparisons
- Risk D: PX4 SITL integration not on the critical path $\Rightarrow$ explicitly framed as future work

9.4 Stage 2 Outlook

Three open problems from Stage 1 to Stage 2:

1. Joint MAP optimization on continuous-time B-splines (U1)

$$\min_{\{c_i\}} \left\{ \underbrace{J_{MAP}^{SLAM}(\{c_i\})}_{\text{factor graph}} + \lambda \underbrace{J_{plan}(\{c_i\})}_{\text{smooth+collision+PI}} \right\}$$

2. Explicit active perception (strong-form U3)

$$J_{active} = - \sum_i \log \det(\Lambda_{pred}(c_i))$$

3. Let the SLAM factor graph absorb planning costs (U2)

Migration path:

- PL Predictor reused, but upgraded from advisory to part of certified path
- σ_{grow} surrogate replaced by full FIM forward propagation
- Predictor frequency drops from 10 Hz to 2 Hz, trading rate for U1 capability
- Learned per-block $P_{fault,b}$, skew/dynamic/symmetry features brought online

9.5 Explicit Requests for Advisor Feedback

Five questions for today's meeting:

1. **Submission target:** For Stage 1, T-IV / ICRA / IROS—which fits best? Should we split into two papers (factor-aware LiDAR ARAIM + PL-Predictor planning architecture)?
2. **Contribution ordering:** Among the three contributions, which should be the headline?
3. **Experimental depth:** Are large-scale demo11 simulations sufficient, or is a real dataset (UrbanNav / M2DGR) required?
4. **Boundary of theoretical proof:** If a closed-form conservatism proof for σ_{grow} is intractable, is Monte-Carlo numerical validation acceptable?
5. **Stage 2 prioritization:** Should all three Stage 2 directions (U1 / U2 / strong-form U3) be pursued, or focus on one as the PhD core contribution?

Thank you for your attention — Q & A